virvelle

CORSO DI MOBILE PROGRAMMING

PROGRAMMA GOL

Virvelle S.r.l. – Società Benefit 2025. All rights reserved. Confidential and proprietary

Finanziato dall'Unione europea
NextGenerationEU

Benvenuti al Laboratorio 4! 💻

Fino ad oggi le nostre app vivevano in una "scatola chiusa". Non sapevano nulla del mondo esterno.
Oggi rompiamo le pareti: la nostra app si collegherà a **Internet** per scaricare dati in tempo reale.

Come funzionano i Laboratori:

1. 🧠 **Briefing & Recap:** 10 minuti di ripasso.
2. 👨‍🏫 **Scaffolding (Insieme):** Scriviamo l'architettura base guidati da me.
3. 🚀 **Sfida Autonoma:** Aggiungete funzionalità da soli (o in team).

Recap Rapido: I 3 Mattoncini di Oggi

Per parlare con Internet, useremo questi 3 concetti:

1. **Retrofit:** 💡 *È il nostro "cameriere". Noi gli diamo l'ordinazione (l'URL), lui va in cucina (il Server), aspetta, e ci riporta il piatto pronto (il file JSON) già trasformato in oggetti Kotlin.*
2. **Asincronia (`LaunchedEffect`):** 💡 *Internet è lento. Se aspettiamo il server sul "Main Thread", l'app si congela. Le chiamate di rete vanno fatte in background!*
3. **Gli Stati della UI (`UI State`):** 💡 *Quando interroghiamo un'API, l'app può trovarsi in soli 3 stati: Caricamento (Rotellina), Successo (Dati) o Errore (Disastro). Dobbiamo gestirli tutti.*

virvelle

Il Progetto di Oggi

🎬 **Movie Finder (Internet & API)**

Il Problema Reale

Un'app come Netflix non ha tutti i film salvati dentro il vostro telefono, altrimenti peserebbe 50 Terabyte! I dati risiedono su un server distante.
Noi dobbiamo imparare a **chiedere** questi dati solo quando l'utente li cerca, mostrando un feedback visivo mentre aspettiamo la risposta.

Architettura dell'App

Costruiremo un'app basata sulle API pubbliche di **OMDB** (Open Movie Database):

- **Il Motore:** Una `data class` per mappare il JSON in arrivo (Titolo, Trama, URL della Locandina).
- **L'Attesa:** Una schermata che mostra un `CircularProgressIndicator` mentre Retrofit lavora.
- **Il Risultato:** Una Card accattivante che mostra i dettagli del film scaricato.

virvelle

Fase 1: Scaffolding

(Lo facciamo insieme)

Cosa faremo ora nella Fase 1

Aprite Android Studio. Io condividerò lo schermo e scriveremo insieme la parte più "noiosa" ma fondamentale del networking.

1. Aggiungeremo il permesso `INTERNET` nel Manifest (senza questo, non si va da nessuna parte).
2. Creeremo le Data Class leggendo la struttura del JSON di OMDB.
3. Configureremo l'istanza di Retrofit.
4. Faremo una chiamata "fissa" (es. cercheremo sempre "Batman") per vedere comparire la rotellina di caricamento e i primi testi.

Preparate le tastiere e verificate di essere connessi al Wi-Fi!

virvelle

Fase 2: Sfida Autonoma

(Tocca a voi)

Il vostro turno

Abbiamo il collegamento a Internet, ma l'app cerca sempre lo stesso film e i dati sono solo testo grezzo. È il momento di farla diventare una vera app!

Avete **tre Task** da completare per finire il laboratorio: Grafica (con un focus sulle immagini), Logica e un Crash Test molto subdolo.

🎨 Task 1: Sfida Grafica (Le Locandine)

Il JSON di OMDB ci restituisce l'URL del poster del film (es. `https://.../poster.jpg`). Ma Compose di base non sa scaricare immagini da un link!

- **Obiettivo:** Mostrare la locandina del film.
- **La Soluzione:** Dovete usare una libreria esterna chiamata **Coil**.
- Cercate su Google (o chiedete all'IA) come aggiungere Coil al file `build.gradle.kts` e come usare il componente `AsyncImage` per disegnare l'immagine a schermo. Poi, abbellite la Card!

⚙️ Task 2: Sfida Logica (Motore di Ricerca)

Cercare sempre "Batman" è noioso. L'utente deve poter scegliere!

Obiettivo: Rendere la chiamata API dinamica.

- Inserite un `OutlinedTextField` e un `Button` ("Cerca") sopra la Card del risultato.
- Salvate il testo digitato dall'utente in una variabile di stato.
- Modificate il `LaunchedEffect` (o usate un `CoroutineScope` sul click del bottone) affinché Retrofit passi alla query dell'API il titolo digitato dall'utente invece della parola fissa.

💥 Task 3: Crash Test (Il Film Inesistente)

Cosa succede se l'utente cerca "asdfghjkl"?

Obiettivo: Gestire le risposte "ingannevoli" dell'API.

- OMDB è subdolo: se non trova un film, non va in errore di rete (Codice 404). Risponde con un **Successo (Codice 200)**, ma il JSON dice: `{"Response":"False", "Error":"Movie not found!"}`.
- Se provate a leggere il Titolo da quel JSON, l'app vi mostrerà dati vuoti o andrà in crash per un valore nullo.
- **La Soluzione:** Aggiornate la vostra `data class` per leggere il campo "Response". Prima di disegnare la UI, verificate con un `if/else` se il film è stato trovato. Altrimenti mostrate un messaggio di errore all'utente!

virvelle

🤖 Un avviso sull'Intelligenza Artificiale

Siete Ingegneri, non Copiatori.

Come usare l'IA nel modo giusto

Lavorare con JSON e Retrofit richiede molta precisione. L'IA è perfetta per aiutarvi a fare il "parsing" (traduzione) dei dati.

❌ **Prompt Sbagliato:** *"Scrivimi l'app Movie Finder con OMDB in Jetpack Compose."*
(Vi darà un codice basato su librerie vecchie di 4 anni che non compilerà mai).

✅ **Prompt Corretto (Da Ingegnere):** *"Come si usa la libreria Coil in Jetpack Compose per caricare un'immagine partendo da una stringa URL? Dammi un esempio di AsyncImage."*

Il ruolo del Copilota

Se l'IA vi suggerisce di usare librerie come `Glide` o `Picasso`, sappiate che sono vecchie e pensate per XML, non per Compose. Usate **Coil**.

Se l'app non compila, copiate l'errore rosso (Logcat) e chiedete all'IA di spiegarvi *cosa* significa quell'errore, non solo di risolverlo ciecamente.

Let's Code! 🚀

Condivido lo schermo. Apriamo il Manifest e chiediamo il permesso a Internet!

virvelle

virvelle

Learn Your Way, Grow With Us.

Virvelle S.r.l. – Società Benefit 2025. All rights reserved. Confidential and proprietary